

Otros Servicios

Tema 8

Prof. Ing. Angel Brito Segura

1. Introducción

Con el auge de la computación en la nube (*cloud computing*), los requisitos y las posibilidades para desarrollar, desplegar y diseñar aplicaciones han cambiado significativamente. Los proveedores de nube ofrecen una variedad de servicios bajo demanda, comenzando con servidores simples (virtuales), redes, almacenamiento, bases de datos y mucho más. Desplegar y gestionar estos servicios debe ser muy práctico, ya sea de forma interactiva o mediante el uso de interfaces de programación de aplicaciones (APIs).

En este contexto, los servicios adicionales que complementan la infraestructura básica son esenciales para mejorar la funcionalidad, la eficiencia y la seguridad de las aplicaciones en la nube. Estos servicios incluyen, entre otros, asignación de puertos, direcciones IP, intermediarios web y orquestación de contenedores a gran escala.

2. RFC 1700

Constituye uno de los documentos históricos fundamentales del desarrollo de Internet, pues recopila los números asignados oficialmente para protocolos, servicios y puertos que permiten la comunicación efectiva entre sistemas heterogéneos. Servía de referencia indispensable para administradores al implementar y configurar sistemas compatibles, como firewalls, servidores y aplicaciones empresariales.

Fue publicado en octubre de 1994 por la Internet Assigned Numbers Authority (IANA) y contenía la lista de números de puerto asignados y bien conocidos para varios servicios y protocolos de Internet. Estos puertos (del 0 al 1023) están reservados para servicios estándar y protocolos ampliamente utilizados, como HTTP (puerto 80), HTTPS (puerto 443), FTP (puerto 21), SMTP (puerto 25), entre otros.

Aunque actualmente las asignaciones se gestionan por la IANA en línea, este RFC sentó las bases y su consulta sigue siendo útil en ambientes heredados y para comprender la evolución de los estándares de red.

3. Servidor Proxy

Este servidor es un intermediario entre un cliente y un servidor. Actúa como un puente que recibe las solicitudes del cliente y las reenvía al servidor correspondiente, y luego devuelve la respuesta del servidor al

cliente. Los servidores proxy se utilizan para diversos propósitos, como mejorar la seguridad, el rendimiento y la privacidad en las comunicaciones en línea.

Existen varios tipos de servidores proxy, cada uno con sus propias características y usos:

- **Proxy HTTP:** Maneja solicitudes HTTP y HTTPS, permitiendo a los usuarios acceder a sitios web a través del proxy.
- **Proxy transparente:** No requiere configuración en el cliente y puede interceptar el tráfico sin que el usuario lo note.
- **Proxy inverso:** Se coloca frente a los servidores web y maneja las solicitudes entrantes, distribuyendo la carga entre varios servidores.
- **Proxy anónimo:** Oculta la dirección IP del cliente, proporcionando privacidad adicional.

Los servidores proxy pueden ofrecer beneficios como el almacenamiento en caché de contenido para mejorar la velocidad de acceso, filtrado de contenido para bloquear sitios web no deseados y anonimato para proteger la identidad del usuario en línea. Sin embargo, también pueden introducir latencia adicional y posibles problemas de seguridad si no se configuran correctamente.

4. Servidor DHCP

El **Protocolo de Configuración Dinámica de Host** (DHCP, por sus siglas en inglés) es un protocolo de red utilizado para asignar direcciones IP y otros parámetros de configuración de red a dispositivos en una red de manera automática, definido en el RFC 2131 para IPv4 y en el 8415 para IPv6. Ocupa el puerto 67 para el servidor y el 68 para el cliente siendo responsable de gestionar y asignar estas direcciones IP a los clientes que se conectan a la red.

Cuando un dispositivo (cliente DHCP) se conecta a una red, envía una solicitud DHCP para obtener una dirección IP y otros parámetros necesarios para comunicarse en la red, como la máscara de subred, la puerta de enlace predeterminada y los servidores DNS. El servidor DHCP responde a esta solicitud asignando una dirección IP disponible del rango configurado y proporcionando los demás parámetros necesarios.

El uso de un servidor DHCP simplifica la administración de redes, ya que elimina la necesidad de configurar manualmente las direcciones IP en cada dispositivo. Además, permite una gestión eficiente del espacio de direcciones IP, evitando conflictos y facilitando la movilidad de los dispositivos dentro de la red.

5. Arquitectura Cloud Native

La idea de esta arquitectura es optimizar el software para la eficiencia de costos, la confiabilidad y un *time-to-market* (tiempo de comercialización) más rápido, utilizando una combinación de patrones de diseño culturales, tecnológicos y arquitectónicos. La **Cloud Native Computing Foundation (CNCF)** define a esta arquitectura como:

“Las tecnologías *cloud native* empoderan a las organizaciones para construir y ejecutar aplicaciones escalables en entornos modernos y dinámicos, tales como nubes públicas, privadas e híbridas. Los contenedores, *service meshes*, los microservicios, la infraestructura inmutable y las APIs declarativas ejemplifican este enfoque.

Estas técnicas habilitan sistemas débilmente acoplados que son resilientes, gestionables y observables. Combinadas con una automatización robusta, permiten a los ingenieros realizar cambios de alto impacto de manera frecuente y predecible con un mínimo de esfuerzo manual repetitivo...”

Las aplicaciones tradicionales o heredadas (*legacy*) suelen estar diseñadas con un enfoque **monolítico**, lo que significa que son autocontenidas e incluyen toda la funcionalidad y los componentes necesarios para cumplir una tarea. Una aplicación monolítica generalmente posee una única base de código y se distribuye como un solo archivo binario capaz de ejecutarse en un servidor.

Aunque desarrollar y desplegar una aplicación en este formato puede ser muy sencillo, puede resultar igualmente difícil gestionar la complejidad, escalar el desarrollo entre múltiples equipos, implementar cambios rápidamente y ser capaz de escalar la aplicación de manera eficiente cuando se encuentra bajo una gran carga de trabajo.

La **arquitectura nativa de la nube** puede ofrecer soluciones para la creciente complejidad de las aplicaciones y la demanda de los usuarios. La idea básica es descomponer la aplicación en piezas más pequeñas, lo que las hace más manejables. En lugar de proporcionar toda la funcionalidad en una sola aplicación, se tienen múltiples aplicaciones **desacopladas** que se comunican entre sí a través de una red. A estas aplicaciones pequeñas e independientes, con un alcance de funciones claramente definido, se les suele denominar **microservicios**.

Esto hace posible tener múltiples equipos, cada uno con la propiedad (*ownership*) de diferentes funciones de la aplicación, pero también operarlas y escalarlas individualmente. Cada microservicio puede ser desarrollado, desplegado, gestionado y escalado de forma independiente, lo que permite una mayor agilidad y flexibilidad en el desarrollo de software.

Esta arquitectura puede tener muchas ventajas, pero también puede ser muy compleja de integrar y sobretodo de orquestar cuando se tienen muchos microservicios. Por ello, existen herramientas y plataformas que facilitan la gestión de aplicaciones nativas de la nube, como Kubernetes, Docker Swarm y OpenShift.

5.1. Orquestación de Contenedores

Proceso automatizado de gestionar, escalar y desplegar aplicaciones en contenedores. Recordar que los contenedores son una forma ligera y portátil de empaquetar aplicaciones junto con sus dependencias, lo que facilita su ejecución en diferentes entornos. Su orquestación se vuelve esencial cuando se trabaja con múltiples de ellos que forman parte de una aplicación más grande, especialmente en entornos de producción.

Las herramientas de orquestación de contenedores proporcionan funcionalidades clave para gestionar el ciclo de vida de los contenedores, incluyendo:

- **Despliegue automático:** Permiten desplegar y actualizar aplicaciones en contenedores de manera automática.
- **Escalabilidad:** Facilitan la escalabilidad horizontal y vertical de las aplicaciones según la demanda.
- **Balanceo de carga:** Distribuyen el tráfico entre los contenedores para optimizar el rendimiento.
- **Recuperación automática:** Detectan fallos en los contenedores y los reinician o reemplazan automáticamente.
- **Gestión del estado:** Mantienen el estado deseado de la aplicación, asegurando que el número correcto de instancias esté siempre en ejecución.

Esta orquestación se vuelve fundamental para gestionar aplicaciones modernas basadas en microservicios, permitiendo a las organizaciones aprovechar al máximo las ventajas de la computación en la nube y mejorar la eficiencia operativa.

5.1.1. Kubernetes

Plataforma de orquestación de contenedores de código abierto altamente popular que puede utilizarse para automatizar el despliegue, el escalado y la gestión de cargas de trabajo contenerizadas.

Originalmente diseñado y desarrollado por Google, Kubernetes se convirtió en código abierto en 2014 y, junto con el lanzamiento de la versión v1.0, fue donado a la recién formada **Cloud Native Computing Foundation (CNCF)** como su primer proyecto. Muchas tecnologías nativas de la nube giran en torno a Kubernetes, ya sean herramientas de bajo nivel como los entornos de ejecución de contenedores (*container runtimes*), monitoreo o herramientas de entrega de aplicaciones.

Kubernetes se utiliza a menudo como un **clúster**, lo que significa que se extiende a través de múltiples servidores que trabajan en diferentes tareas para distribuir la carga de un sistema. Esta es una decisión de diseño inicial basada en los requisitos de Google, donde miles de millones de contenedores se inician cada semana. Dada la alta escalabilidad vertical de Kubernetes, es posible tener clústeres con miles de nodos de servidor, a través de múltiples centros de datos y regiones.

Desde una perspectiva de alto nivel, los clústeres de Kubernetes constan de dos tipos diferentes de nodos de servidor que conforman el clúster:

Nodos del plano de control (*Control plane nodes*):

Son el cerebro de la operación. Estos nodos contienen varios componentes que gestionan el clúster y controlan diversas tareas como el despliegue, la planificación y la auto-reparación de las cargas de trabajo en contenedores.

Nodos de trabajo (*Worker nodes*):

Es donde se ejecutan las aplicaciones en el clúster. Este es el único trabajo de estos nodos y no tienen ninguna lógica adicional implementada. Su comportamiento está completamente controlado por el nodo del plano de control.

Referencias

- [1] B. A. Forouzan, *Data Communications and Networking*. McGraw-Hill, 2003.
- [2] D. E. Comer, *Internetworking with TCP/IP Vol. 1: Principles, Protocols, and Architecture*. Prentice Hall, 2000.
- [3] C. Hunt, *TCP/IP Network Administration*. O'Reilly Media, 2002.
- [4] J. Edwards y R. Bramante, *Networking Self-Teaching Guide OSI, TCP/IP, LANs, MANs, WANs, Implementation, Management, and Maintenance*. Wiley, 2009.
- [5] M. Burgess, *Principles of Network and System Administration*. Wiley, 2004.